

Lovci pokladov

Časový limit: 8 s Limit pamäte: 256 MiB

Hľadáš dávno stratený poklad na obrovskom poli buniek s rozmermi $N \times N$ pomocou čarovného kompasu. Na navzájom rôznych bunkách poľa je ukrytých dokopy $K \leq 3$ truhlíc s pokladom. Tvoj cieľ je jednoduchý: všetky ich nájdi!

Keď položíš kompas na jednu z buniek, nájde najkratšiu cestu k truhlici s pokladom pomocou pohybov doľava, hore, doprava a dole (t. j. k najbližšej truhlici podľa Manhattanovskej vzdialenosti). Potom ukáže smer prvého kroku. Ak existujú viaceré smery, ktorými môže začínať nejaká najkratšia cesta k niektorej najbližšej truhlici, kompas vráti všetky tieto smery. Ak bunka obsahuje truhlicu, kompas to oznámi namiesto smeru.

Vždy keď nájdeš truhlicu s pokladom, vyrabuješ jej obsah, ale nemôžeš ju odstrániť - je príliš ťažká. Tvoj kompas nevie, či je truhlica plná alebo prázdna. Bude ukazovať smer k najbližšej truhlici, či už je prázdna alebo plná.

Pokús sa nájsť polohu všetkých truhlíc s pokladom na čo najmenší počet použítí kompasu.

Úloha

Toto je interaktívna úloha. V každom testovacom prípade (t.j. v každom spustení tvojho programu) bude musieť tvoj program vyriešiť niekoľko samostatných dobrodružných výprav. S hodnotiacim systémom by si mal komunikovať pomocou knižnice poskytnutej organizátormi. Táto knižnica obsahuje nasledujúce deklarácie:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — zavolaním tejto funkcie začneš ďalšiu výpravu za pokladom. Nastaví premennú *N* na veľkosť mriežky a *K* na počet truhlíc. Ak v aktuálnom behu programu už nie je ďalšia výprava, čo riešiť, funkcia nastaví *N* a *K* na -1 ; v takom prípade musíš ukončiť program s návratovým kódom 0. Všimni si, že túto funkciu môžeš zavolať aj bez toho že by si v aktuálnej výprave našiel všetky truhlice, napr. ak sa snažíš získať iba čiastočné body.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — ide o konštanty používané v návratových hodnotách funkcie *Query* (pozri nižšie).
- **int** *Query*(**int** *x*, **int** *y*) — ak bunka na súradniciach (*x*, *y*) obsahuje truhlicu, táto funkcia vráti *TREASURE*. Inak vráti súčet jednej alebo viacerých konštánt *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* a *DIR_DOWN*, ktoré určujú, aké pohyby kompas ukáže pri umiestnení do bunky (*x*, *y*). Súradnice *x* a *y* musia byť celé čísla od 0 do $N - 1$. (Poznámka: v tejto úlohe súradnice *y* stúpajú zhora nadol.)

Po tom, čo *NextHunt* vráti $N = K = -1$, tvoj program už nesmie volať ani *NextHunt*, ani *Query*. Taktiež nesmie volať *Query* pred prvým volaním *NextHunt*. Ak tvoj program tieto obmedzenia nedodrží, alebo ak vykoná viac než 1000 dotazov počas jednej výpravy, alebo ak pri volaní *Query* použije hodnoty *x* a/alebo *y* mimo povoleného rozsahu, knižnica ukončí tvoj program a vráti verdikt run-time-error (RTE) pre aktuálny testovací prípad.

Truhlica sa považuje za nájdenú, ak si sa aspoň raz pozrel na bunku, ktorá ju obsahuje. Ak tvoj program zavolá *NextHunt* predtým než nájde všetky truhlice v aktuálnej výprave, nepovažuje sa to za chybu, ale ovplyvní to tvoje skóre (viac o hodnotení nižšie).

Na prístup ku knižnici by mal tvoj program includnúť hlavičkový súbor `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Tento hlavičkový súbor si môžeš stiahnuť tu: `treasurehuntlib.h`.

Na pomoc pri vývoji riešenia je dostupná jednoduchá implementácia knižnice tu: `treasurehuntlib-public.cpp`. Ak ju chceš skompilovať spolu so svojím programom, jednoducho pridaj jej názov ako parameter kompilátora, napr.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

kde `foo.cpp` je názov súboru obsahujúceho tvoje riešenie.

Implementácia v súbore `treasurehuntlib-public.cpp` podporuje okrem vyššie uvedených deklarácií aj ďalšiu funkciu s názvom **`void InitFromFile(const char *fileName)`**, ktorá načíta zoznam výprav zo súboru a umožní vám hrať hru s nimi namiesto generovania vlastných náhodných hľadání pokladov. Viac podrobností nájdete v samotnom súbore `treasurehuntlib-public.cpp`.

Na hodnotiacom serveri bude použitá iná implementácia knižnice, preto by si nemal nič predpokladať o tom, ako presne funguje. Môžeš však predpokladať, že nešpiní globálny namespace žiadnymi deklaráciami okrem vyššie uvedených (*NextHunt*, *Query* a tých piatich konštánt).

Tvoj program nesmie čítať zo štandardného vstupu ani zapisovať na štandardný výstup, pretože tie budú použité našou implementáciou knižnice na hodnotiacom serveri na komunikáciu so zvyškom hodnotiaceho prostredia.

Vstup

Obmedzenia

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Počas jedného spustenia programu bude najviac 100 000 dobrodružných výprav.
- Počas jednej výpravy môžeš vykonať najviac 1000 dotazov.
- Hodnotiaci systém nie je adaptívny.

Podúlohy

- Podúloha 1 (10 bodov) $K = 1$
- Podúloha 2 (30 bodov) $K = 2$
- Podúloha 3 (60 bodov) $K = 3$.

Hodnotenie

Podúloha môže pozostávať z viacerých testovacích prípadov (viacerých spustení tvojho programu) a každý testovací prípad môže obsahovať viacero výprav. Na účely hodnotenia sa všetky výpravy v rámci danej podúlohy vyhodnocujú spoločne bez ohľadu na to, ako boli pôvodne rozdelené medzi testovacie prípady. Pre i -tu výpravu označme veľkosť mriežky ako $N_i \times N_i$, počet truhlíc ako K_i , počet dotazov vykonaných programom ako Q_i a počet nájdených truhlíc ako F_i . Ďalej označme S maximálny počet bodov získateľných za túto podúlohu. Počet bodov, ktoré dostane program, sa vypočíta takto:

- Ak tvoj program vždy našiel všetky truhlice (t. j. ak $F_i = K_i$ pre všetky i), jeho skóre závisí od $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ bodov, kde } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Ak tvoj program nenašiel vždy všetky truhlice (t. j. existuje i také, že $F_i < K_i$), dostane

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ bodov.}$$

Inými slovami, polovicu bodov získaš za to že si našiel všetky truhlice, a druhú polovicu za to že ti stačilo čo najmenej dotazov. Na získanie plného počtu bodov by malo tvoje riešenie nájsť všetky truhlice za najviac $11 \lceil \log_2 N_i \rceil$ dotazov. Medzi $11 \lceil \log_2 N_i \rceil$ a $20 \lceil \log_2 N_i \rceil$ dotazmi skóre lineárne klesá; a ak tvoje riešenie použije viac než $20 \lceil \log_2 N_i \rceil$ dotazov, získa iba prvú polovicu bodov za to že našlo všetky truhlice. (Symboly $\lceil \cdot \rceil$ znamenajú, že hodnota $\log_2 N_i$ sa zaokrúhľuje nahor na najbližšie celé číslo.)

Ak vyššie uvedené vzorce vypočítajú neceločíselný počet bodov, výsledok sa zaokrúhli na najbližšie celé číslo.

Ak tvoj program skončí s chybou počas behu alebo nedodrží vyššie uvedený protokol používania knižnice, získa 0 bodov za celú podúlohu. Na získanie čiastočných bodov za nájdenie iba podmnožiny truhlíc musí tvoj program preto korektne ukončiť výpravu zavolaním *NextHunt*.

Príklad

Volanie	Výsledok
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$